

Lab1:DataLab

25303050208 刘瀚文

1. 输出最高位是 1，剩余全是 0，所以可以直接输出 $1 \ll 31$
2. 要实现 $x \wedge y$ ，就是异或，相同为 0，不同为 1，所以可以直接写出他的逻辑公式，替换 2
3. 先给 $x \gg 31$ ，正数就都是 0，负数就都是 1，然后和 x 的反做 & 运算，负数就是相反数，正数都是 0
4. 首先将源字节编号 src 左移 3 位换算成比特偏移量，把 x 右移该偏移量后与 $0xFF$ 做按位与，提取出 src 位置的单个字节；接着把目标字节编号 dst 左移 3 位得到目标比特偏移，将提取到的字节左移至 dst 对应的位置；然后构造掩码，将 $0xFF$ 左移到 dst 位置后取反，用这个掩码和 x 做按位与，把 x 中 dst 字节的原有内容清零、其余字节保持不变；最后将清零后的 x 与移位到位的源字节做按位或运算，把源字节填入 dst 位置，得到最终结果。
5. 先对 x 做算术右移 n 位，再构造掩码将结果左侧 n 个比特置 0，其余比特保留；将算术右移的结果和掩码做按位与，消除算术右移补在高位的 1，实现高位补 0 的逻辑右移。
6. 取出 x 的低 4 位和高 4 位，将低 4 位左移 4 位，再与右移 4 位得到的高 4 位做按位或，完成一字节内高低 4bit 互换。
7. 先对 x 按位取反，把找 0 位转为找 1 位；求出取反后数字最低的 1 比特，清除该比特，再求出剩下数字最低的 1 比特，就是原数字倒数第二个 0 比特对应的掩码；如果没有 0 比特，结果为 0。
8. 通过多轮折叠异或，依次将高 16 位、高 8 位、高 4 位、高 2 位、高 1 位异

或至低位，把全部 32 位比特中 1 的数量的奇偶信息压缩至最低比特， $x \& 1$ 得到 1 代表 1 的个数为奇数，!对结果取反，最终实现奇校验功能。

9. 循环右移就是先把 n 模 32 得到 s ， s 是 0 就返回原值。把 x 转成无符号数，右移 s 位，同时把低 s 位左移 $32-s$ 位补到高位，两部分或起来再转回 `int`。转无符号是为了防止右移时前面补 1。
10. 这个函数就是把 x 拆成高位的商和低 n 位的余数，余数超过一半就进位，刚好等于一半时看商是不是奇数，奇数才进位、偶数就不动，最后把低 n 位清零
11. 先算出向下取整的中点，再判断是否需要加一来向 x 靠拢。为避免 $x+y$ 溢出，用位运算 $(x \& y) + ((x \wedge y) \gg 1)$ 等价计算向下取整的中点；接着，只有当 x 和 y 一奇一偶时（即 $(x \wedge y) \& 1$ 为真），中点才会有小数部分，此时需要判断 $x \geq y$ ，若是，则向下取整会偏离 x ，需加一。由于直接相减 $x - y$ 可能溢出，需分情况处理：当 x 和 y 异号时， $x \geq y$ 等价于 x 非负；同号时， $x - y$ 不会溢出， $x \geq y$ 等价于 $x - y$ 非负。最后将“有小数”与“ $x \geq y$ ”这两个条件相与，若同时成立则给中点加一，否则直接返回。
12. 先判断 a 和 b 谁大谁小，确定区间的最小值 `min` 和最大值 `max`，然后再检查 x 是否落在 `min` 和 `max` 之间。它判断大小的方法是用异或看符号是否相同，再通过减法 $a - b$ 的符号位来判断顺序，最后用掩码把 a 和 b 交换成 `min` 和 `max`。
13. 这个函数把 $x5$ 拆成 x 左移 2 位再加 x ，然后分两步判断溢出：先看 x 左移 2 位会不会溢出，用 x 右移 29 位和 x 右移 31 位异或来判断；再看相加后结果的符号位和 x 是否相同，不同就说明加法溢出了。两个溢出

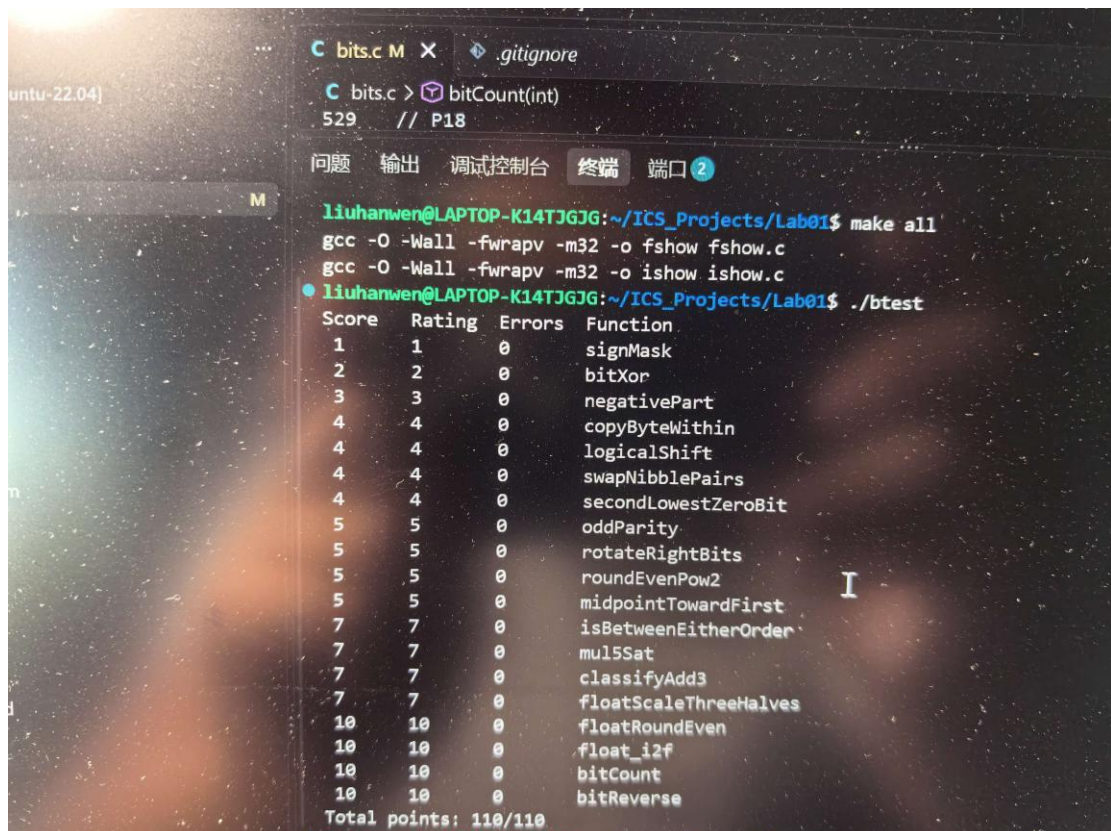
条件或起来，只要有一个溢出，就按 x 的正负返回 `INT_MAX` 或 `INT_MIN`，否则返回正常的 $x5$ 结果。

14. 先算 $s1 = x + y$ 。如果 x 、 y 都是正数，但 $s1$ 变成了负数，这一步正溢出，记 $+1$ ；如果 x 、 y 都是负数，但 $s1$ 变成了非负，这一步负溢出，记 -1 ；否则记 0 。再用 $s1 + z$ 算 $s2$ 。同样判断 $s1$ 和 z 是否同号且结果异号，正溢出记 $+1$ ，负溢出记 -1 。把两步的溢出量相加直接返回。
15. 这函数就是把 `float` 拆成符号、阶码、尾数，先特判掉无穷、NaN 和 0 。
非规格化时尾数没有隐含的 1 ，乘 1.5 就是乘 3 再除 2 ，所以 $T = m + (m < 1)$ 再右移一位；舍入看丢的位和末位是不是都为 1 ，是就进一。进位到 `0x800000` 就变成规格化，阶码置 1 。规格化时补上隐含的 1 ，同样乘 3 除 2 ，舍入进位如果到了 `0x1000000`，阶码加 1 ，尾数截掉最高位。最后把符号、阶码、尾数拼回去返回。
16. 先把 `uf` 拆成符号、阶码和尾数，特判掉无穷、NaN、绝对值小于 0.5 、正好 0.5 、大于 0.5 这几种边界；对于大于等于 1 的情况，算出小数部分占多少位，取出舍入位、粘滞位和整数最低位，按银行家舍入法判断是否进位；进位后如果尾数溢出就右移一位并让阶码加一，最后把符号、阶码和尾数拼回去返回。
17. 先把整数转成无符号绝对值，符号位单独存；找到最高位得到阶码；如果阶码不大于 23 就直接左移对齐，否则右移并做四舍五入，注意进位到 24 位时阶码加一并右移；最后拼符号、阶码和尾数返回。
18. 先用 `mask1` 统计每 2 位里 1 的个数，接着两两合并，用 `mask2` 统计每 4 位的个数，然后依次用 `mask3`、`mask4`、`mask5` 把统计范围扩大到 8

位、16 位、32 位。最终所有 1 的个数都累加到了 count 的最低 6 位里，直接返回就行。

19. 先在第一步两两交换相邻的 1 位，接着第二行以 2 位为单位交换，后面依次扩展到 4 位、8 位、16 位，最后一行直接交换高低 16 位，每一行都是拿掩码把对应块里的位抠出来，再移位到对称位置合并，几行位运算就完成了整个 32 位整数的翻转。

20. 除 1 到 8 题完全没有使用 AI，9 到 19 均部分使用 AI 加以帮助



```
bits.c M X .gitignore
bits.c > bitCount(int)
529 // P18

问题 输出 调试控制台 终端 端口 2

liuhanwen@LAPTOP-K14TJGJG:~/ICS_Projects/Lab01$ make all
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
liuhanwen@LAPTOP-K14TJGJG:~/ICS_Projects/Lab01$ ./btest
Score Rating Errors Function
1 1 0 signMask
2 2 0 bitXor
3 3 0 negativePart
4 4 0 copyByteWithin
4 4 0 logicalShift
4 4 0 swapNibblePairs
4 4 0 secondLowestZeroBit
5 5 0 oddParity
5 5 0 rotateRightBits
5 5 0 roundEvenPow2
5 5 0 midpointTowardFirst
7 7 0 isBetweenEitherOrder
7 7 0 mul5Sat
7 7 0 classifyAdd3
7 7 0 floatScaleThreeHalves
10 10 0 floatRoundEven
10 10 0 float_i2f
10 10 0 bitCount
10 10 0 bitReverse
Total points: 110/110
```

```
10    10    0    bitCount
10    10    0    bitReverse
```

Total points: 110/110

liuhanwen@LAPTOP-K14TJGJG:~/ICS_Projects/Lab01\$./check_ops.py bits.c

signMask: 1/2 operators

bitXor: 8/8 operators

negativePart: 4/6 operators

copyByteWithin: 9/12 operators

logicalShift: 6/20 operators

swapNibblePairs: 13/18 operators

secondLowestZeroBit: 8/8 operators

oddParity: 12/56 operators

rotateRightBits: 11/16 operators

roundEvenPow2: 23/24 operators

midpointTowardFirst: 19/32 operators

isBetweenEitherOrder: 46/48 operators

mul5Sat: 21/30 operators

classifyAdd3: 26/52 operators

floatScaleThreeHalves: 41/60 operators

floatRoundEven: 31/65 operators

float_i2f: 32/40 operators

bitCount: 23/40 operators

bitReverse: 32/34 operators

All 19 functions passed operator checks.

liuhanwen@LAPTOP-K14TJGJG:~/ICS_Projects/Lab01\$

0 2

行 551, 列 2. 空格 2.



搜索

